Database Applications Lecture 2: Indexing

Santha Sumanasekara

March 2026

RMIT UNIVERSITY

1

1

Announcements

- Assignment 1 – released on Wednesday
- SQL Server – Do you have any issues?
  - Technical Drop-in/ Consultation Session Monday 3:00PM

RMIT UNIVERSITY

2

2

Database Indexing

- Overview – Why indexing is important?
- Examples of index-based queries -- Demos
- Different indexing methods used in databases
  - Clustered vs. Non-clustered indexes
  - Unique vs. Non-unique indexes
- How indexes are built using B+ Trees
- Disadvantages of Indexing

RMIT UNIVERSITY

3

3

Why is database optimisation important?

- As data size grows, query performance degrades.
  - In many applications, growth is not even linear. Recall the exponential growth of Facebook users or Amazon catalog.
- Hardware improvements often lag the growth of data (do not expect exponential growth of SSD (solid state disk) speeds over next decade.)
- Disk Access time is critical!

RMIT UNIVERSITY

4

4

Disk Access!

- First rule of database performance:
  Disk access is the most expensive thing databases do!
- Accessing data in memory can be 10-100ns
- Accessing data on disk can be up to 10s of ms
  - *That's 5-6 orders of magnitude difference!*
  - Even solid-state drives are 10s-100s of µs (1000x slower)
- Unfortunately, disk IO is usually unavoidable
  - Usually the data simply doesn't fit into memory
  - The data needs to be persistent for when the DB is shut down, or when the server crashes, etc.
- DBs work very hard to minimise the amount of disk IO.

RMIT UNIVERSITY

5

5

Disk Access!

This table has 7 million rows – on-time performance of US flights in a calendar year.

- Run the following query twice:

```
SELECT COUNT(*)
   FROM ontime;
```

- Did you see any difference in runtime?
  - Why?

Results (1) **Messages**

13:18:33 Started executing query at Line 1
(1 row affected)
Total execution time: 00:00:00.884

Results (1) **Messages**

13:19:16 Started executing query at Line 1
(1 row affected)
Total execution time: 00:00:00.152

RMIT UNIVERSITY

6

6

Minimising Disk Access!

- Consider the following query:

```
SELECT * FROM ontime WHERE serialnum = 12345;
```

- There are 2 different ways that SQL could find the results:
  - **Do a "full table scan":** look at every single row in the table, return the matching rows.
  - **Make a copy of the table sorted by `serialnum`**, then do a "**binary search**" to find the row where the `serialnum` is 12345.

RMIT UNIVERSITY

7

7

Minimising Disk Access!

Will show "how" later!

- Consider the following query:

```
SELECT * FROM ontime WHERE serialnum = 12345;
```

- Which one is faster?
- It depends on the data, and on how often the query will be executed.
- If the table is only 10 rows long, then a full table scan only requires looking at 10 rows, and the first plan would work out well.
- However our `ontime` table has 7 million rows!
- It would be faster to do a binary search on a sorted table - we only need 23 lookups to find a value in 7 million rows.
- But, sorting of 7 million rows is expensive!

RMIT UNIVERSITY

8

8

Minimising Disk Access!

- Consider the following query:

```
SELECT * FROM ontime WHERE serialnum = 12345;
```

- Apparently, both methods are not very efficient!
- Building and using an **Index** is a very promising approach this problem.

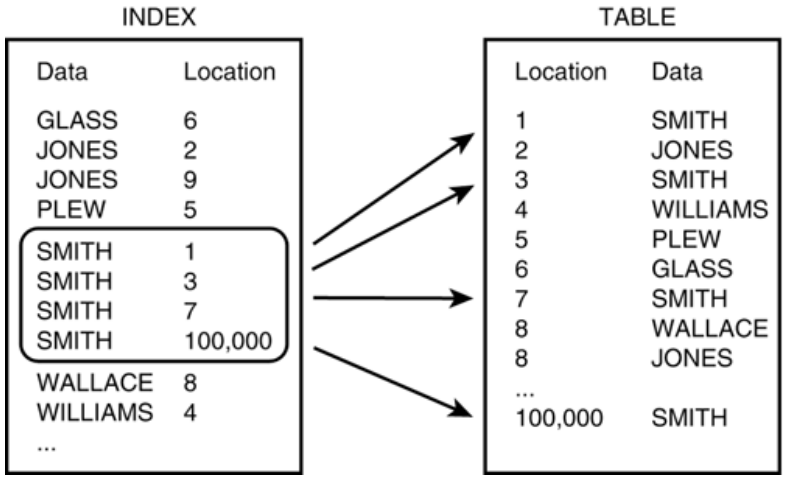
RMIT UNIVERSITY

9

9

Minimising Disk Access!

- An (over-)simplified illustration of how indexing help reduce disk accesses.



RMIT UNIVERSITY

10

10

Minimising Disk Access!

➢ For mathematically inclined:

Given that an index record contains only the indexed field and a pointer to the original record, it will be smaller than the multi-field record that it points to. So the index itself requires fewer disk blocks than the original table, which therefore requires fewer block accesses to iterate through. The schema for an index on the *SerialNum* field is outlined below;

Fieldname	Data type	size
SerialNum	serial	4 bytes
(record pointer)	Special	4 bytes

Note: Pointers in MySQL are 2, 3, 4 or 5 bytes in length depending on the size of the table.

Given our sample database of r = 7,000,000 records with an index record length of R = 8 bytes and using the default block size B = 1,024 bytes. The blocking factor of the index would be bfr = (B/R) = 1024/8 = 128 records per disk block. The total number of blocks required to hold the index is N = (r/bfr) = 7000000/128 = 54688 blocks.
Now a search using the *SerialNum* field can utilize the index to increase performance. This allows for a binary search of the index with an average of log2 54688 = 15.73 = 16 block accesses. To find the address of the actual record, which requires a further block access to read, bringing the total to 16 + 1 = 17 block accesses, a far cry from the 7,000,000 block accesses required to find a *SerialNum* match in the non-indexed table.

RMIT UNIVERSITY

12

12

Demo Time!

Let's try out a query with and without an index to help us.

RMIT UNIVERSITY

13

Demo Time!

- Consider the following query:

```
SELECT * FROM ontime
   WHERE serialnum = 12345;
```

```
SELECT * FROM ontime
   WHERE tailnum ='N305SW';
```

- There are no indexes on any of the columns. So, both queries will require full table scan.

RMIT UNIVERSITY

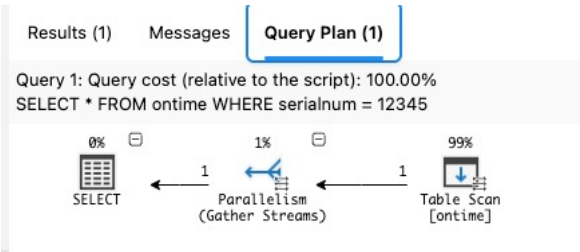
14

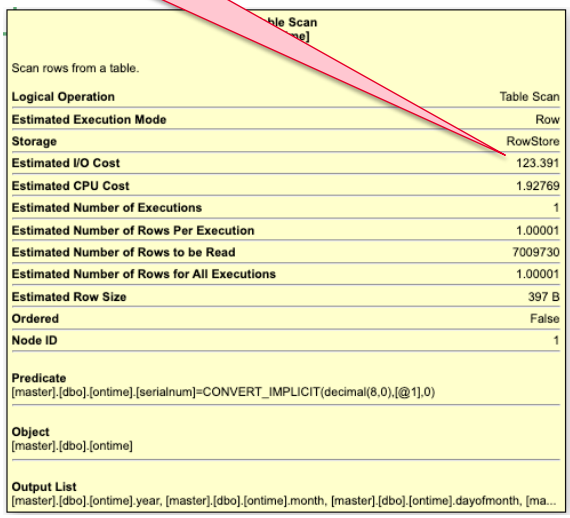
14

Demo Time!

Note the massive I/O cost due to full table scan

- Look at the query plan and estimated costs:





Scan rows from a table.

Logical Operation	Table Scan
Estimated Execution Mode	Row
Storage	RowStore
Estimated I/O Cost	123.391
Estimated CPU Cost	1.92769
Estimated Number of Executions	1
Estimated Number of Rows Per Execution	1.00001
Estimated Number of Rows to be Read	7009730
Estimated Number of Rows for All Executions	1.00001
Estimated Row Size	397 B
Ordered	False
Node ID	1

Predicate
[master].[dbo].[ontime].[serialnum]=CONVERT_IMPLICIT(decimal(8,0),[@1],0)

Object
[master].[dbo].[ontime]

Output List
[master].[dbo].[ontime].year, [master].[dbo].[ontime].month, [master].[dbo].[ontime].dayofmonth, [ma...

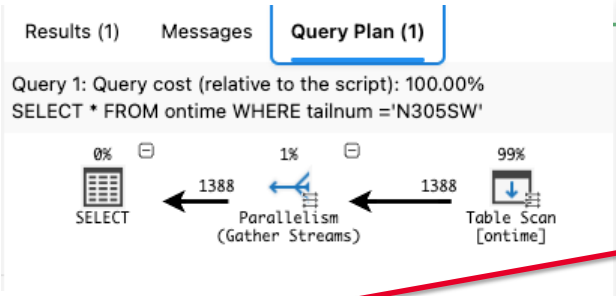
RMIT UNIVERSITY

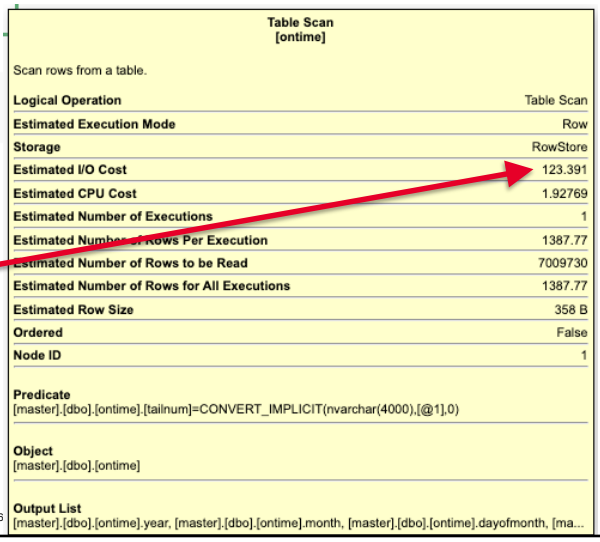
15

15

Demo Time!

- Look at the query plan and estimated costs with the second query:





- Took the same time to process as the previous query, even if the result set has more rows.

RMIT UNIVERSITY

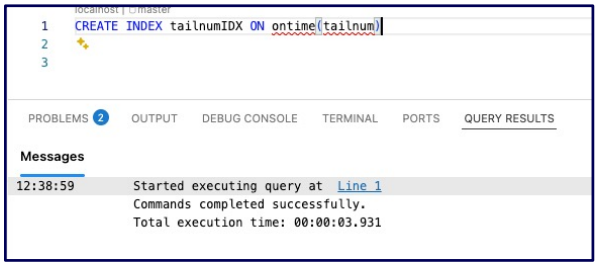
16

16

Let's build an index!

- Build an index on talinum column

```
CREATE INDEX tailnumIDX
    ON ontime(talinum);
```



- There is a substantial one-off cost on building the index.
- If the table is very dynamic (i.e. lot of ongoing insertions and deletions), there is an associated cost of maintaining the index, too.

RMIT UNIVERSITY

17

17

Demo Time!

- Consider the following query:

```
SELECT * FROM ontime
   WHERE tailnum ='N305SW';
```

- Now, there is an index on talinum attribute.
- So, theoretically, this query should utilise the index.

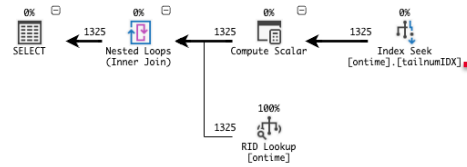
RMIT UNIVERSITY

18

18

Results (1) Messages **Query Plan (1)**

Query 1: Query cost (relative to the script): 100.00%
SELECT * FROM ontime WHERE tailnum ='N305SW'



Index Seek
[ontime].[tailnumIDX]

Scan a particular range of rows from a nonclustered index.

Logical Operation	Index Seek
Estimated Execution Mode	Row
Storage	RowStore
Estimated I/O Cost	0.0060084
Estimated CPU Cost	0.0016145
Estimated Number of Executions	1
Estimated Number of Rows Per Execution	1325.03
Estimated Number of Rows to be Read	1325.03
Estimated Number of Rows for All Executions	1325.03
Estimated Row Size	30 B
Ordered	True
Node ID	4

Object
[master].[dbo].[ontime].[tailnumIDX]

Output List
Bmk1000, [master].[dbo].[ontime].tailnum

Seek Predicates
Seek Keys[1]: Prefix: [master].[dbo].[ontime].tailnum = Scalar Operator(N'N305SW')

RID Lookup
[ontime]

RID Lookup

Logical Operation	RID Lookup
Estimated Execution Mode	Row
Storage	RowStore
Estimated I/O Cost	0.003125
Estimated CPU Cost	0.0001581
Estimated Number of Executions	1325.03
Estimated Number of Rows Per Execution	1
Estimated Number of Rows for All Executions	1325.03
Estimated Row Size	345 B
Ordered	True
Node ID	6

Object
[master].[dbo].[ontime]

Output List
[master].[dbo].[ontime].year, [master].[dbo].[ontime].month, [master].[dbo].[ontime].dayofmonth, [ma...

Seek Predicates
Seek Keys[1]: Prefix: Bmk1000 = Scalar Operator((Bmk1000))

- The total I/O cost is now only a fraction of the original cost.

19

19

Demo Time – What is an Index-only Scan query?

```
SELECT tailnum FROM ontime
   WHERE tailnum LIKE 'N305%';
```

- Many databases support *index-only scans*, which can answer queries from an index alone without any table access.
- Oracle and SQL Server can do index-only scans.
- Some other DBMS's (such as PostgreSQL) has partial support (depends on index type).

RMIT UNIVERSITY

20

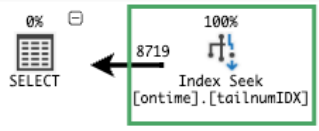
20

Demo Time – What is an Index-only Scan (or index seek) query?

```
SELECT tailnum FROM ontime
   WHERE tailnum LIKE 'N305%';
```

Results (1) Messages **Query Plan (1)**

Query 1: Query cost (relative to the script): 100.00%
SELECT tailnum FROM ontime WHERE tailnum LIKE 'N305%'



Index Seek
[ontime].[tailnumIDX]

Scan a particular range of rows from a nonclustered index.

Logical Operation	Index Seek
Estimated Execution Mode	Row
Storage	RowStore
Estimated I/O Cost	0.0268287
Estimated CPU Cost	0.0097474
Estimated Number of Executions	1
Estimated Number of Rows Per Execution	8718.57
Estimated Number of Rows to be Read	8718.57
Estimated Number of Rows for All Executions	8718.57
Estimated Row Size	61 B
Ordered	True
Node ID	0

Predicate
[master].[dbo].[ontime].[tailnum] like N'N305%'

Object
[master].[dbo].[ontime].[tailnumIDX]

Output List
[master].[dbo].[ontime].tailnum

Seek Predicates
Seek Keys[1]: Start: [master].[dbo].[ontime].tailnum >= Scalar Operator(N'N305'), End: [master].[dbo...

RMIT UNIVERSITY

21

21

Let's explore!

It seems like indexing is such a great idea.

RMIT UNIVERSITY

22

Indexing – To be used to not to be used?

- As you have seen during the demo, there are three possible scenarios with use of indexes:
  - **Index Scan**: when query optimiser determines that it is likely to be efficient to use any available index to reach out to relevant rows in the table;
  - **Index-Only Scan**: when the query can be answered by just only accessing the index. The basic idea is to return values directly out of each index entry instead of consulting the associated table.
  - **Full Table Scan**: Even when an index presents, sometimes the optimiser determines that it is quicker to retrieve data directly from the table, bypassing the index.

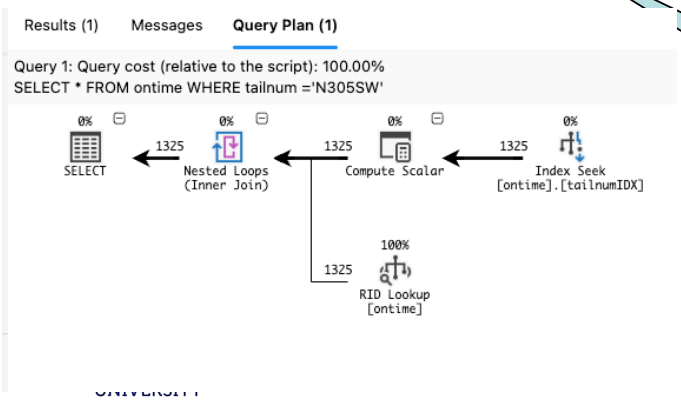
RMIT UNIVERSITY

23

23

Example 1: Index Scan

```
SELECT * FROM ontime
   WHERE tailnum ='N305SW';
```



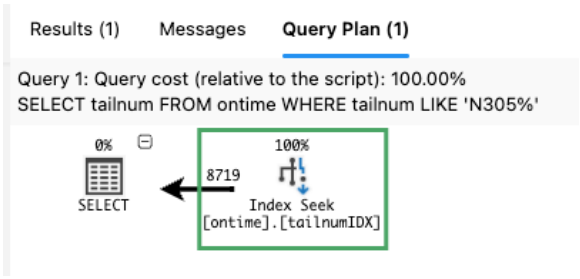
This exact match query benefits from the **`tailnumIDX`**. Cost-based optimizer determines that only a small fraction of **`tailnum`** will meet the WHERE condition, and index scan is quicker than a full table scan

24

24

Example 2: Index-only Scan query

```
SELECT tailnum FROM ontime
   WHERE tailnum LIKE 'N305%';
```



This partial match query only requires **`tailnum`** column, and there is an index on that column: **`tailnum_index`**. So, cost-based optimizer determines that the whole query can be processed on the index, itself and never reaches out to the table.

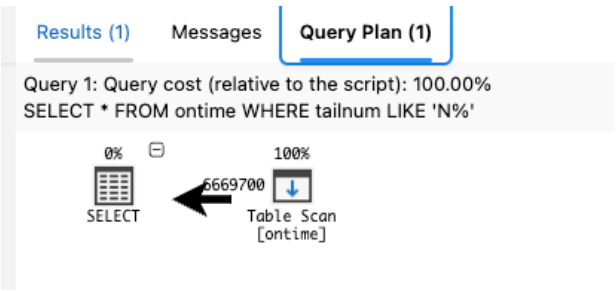
RMIT UNIVERSITY

25

25

Example 3: Full Table Scan query

```
SELECT * FROM ontime
   WHERE tailnum LIKE 'N%';
```



Cost-based optimizer compares the cost of executing the query with and without index. Because there is a large proportion of rows that meets this partial condition, it determines that a full table scan is quicker than index-scan. So, it ignores the index and do a full table scan.

RMIT UNIVERSITY

26

26

Building Indexes -- SQL

- Consider building an index on `SerialNum` attribute.

```
CREATE INDEX serial_index ON ontime(serialnum);
```

- "serial_index" is a label given to the index.
- It is possible to build an index on more than one attribute (a composite index).
- Say if we do lot of queries on dates, we can build a date index.

```
CREATE INDEX date_index ON
                    ontime(year, month, dayofmonth);
```

RMIT UNIVERSITY

27

27

Index Characteristics: Clustered vs. Non-clustered

- Microsoft SQL Server is one of the only few DBMS'es that implement **Clustered Indexes**.
- Clustered indexes sort and store the data rows in the table based on their key values.
- There can only be one clustered index per table.
- When you CREATE TABLE with PRIMARY KEY constraint, it will create a cluster index on the primary key column (in SQL Server only)
- Or else, you can define a clustered index as follows:

```
CREATE CLUSTERED INDEX OntimeSerialnumIDX
    ON ontime(serialnum);
```

RMIT UNIVERSITY

28

28

Clustered Indexes

- Useful when queries
  - With WHERE clauses with BETWEEN, >, >=, <, and <=
  - Return large result sets
  - Use JOINs (e.g. PK with FK type joins)
  - ORDER BY
  - GROUP BY

RMIT UNIVERSITY

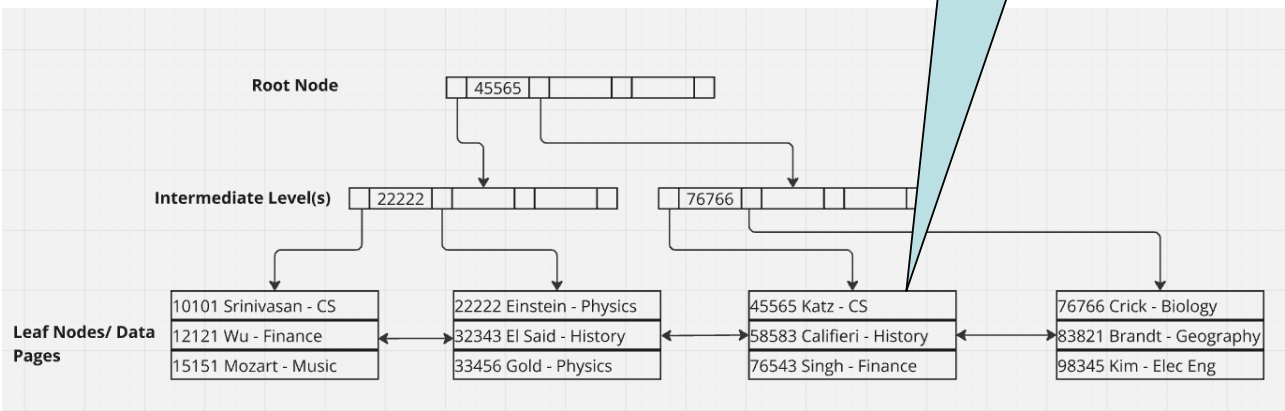
29

29

Clustered Indexes

You can imagine that the leaf level of the index tree is in fact the data pages (i.e. table) itself.

- E.g.: https://miro.com/app/board/uXjVKuTFwdk=/



30

30

Non-clustered Indexes

- Since you can have only one clustered index, every other index must be defined as a non-clustered index
- The leaf level of a non-clustered index is made up of index pages instead of data pages
- If the table is a heap, which means it doesn't have a clustered index, the row locator in leaf nodes is a pointer to the corresponding data pages (in the table) – refer to the diagram in the next slide.

```
CREATE INDEX OntimeTailnumIDX
    ON ontime(tailnum);
```

RMIT UNIVERSITY

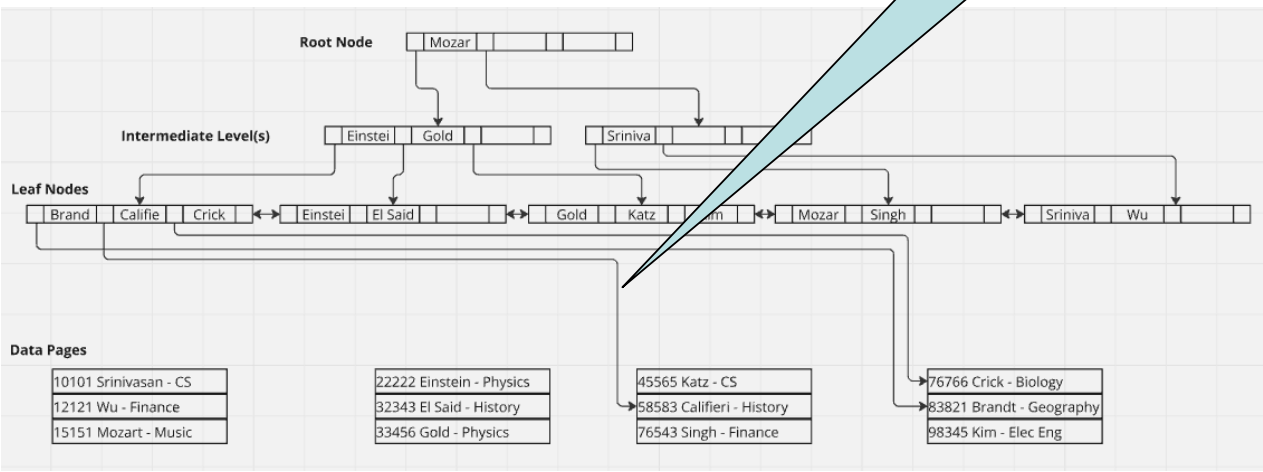
31

31

Non-Clustered Indexes

Pointers in leaf nodes points to corresponding pages where the data rows are located

- E.g.: https://miro.com/app/board/uXjVKuMTHZc=/



32

32

Index Characteristics: Unique vs. Non-unique

- A unique index guarantees that the index key contains no duplicate values
- Specifying a unique index makes sense only when uniqueness is a semantic characteristic of the data itself.
- For example, in ontime table, Serial number is unique while tail number is not unique (each aircraft is used in many many flights).
- When you CREATE TABLE with PRIMARY KEY constraint, it will create a Unique Index on the primary key column (in SQL Server only)
- Or else, you can explicitly create a unique index:

```
CREATE UNIQUE INDEX OntimeSerialnumIDX
    ON ontime(serialnum);
```

33

Index Implementations

- Virtually all databases provide ordered indexes, using some kind of balanced tree structure
  - B_+-tree and B-tree indexes, typically referred to as "b tree" indexes
- Some databases also provide hash indexes
  - More complex to manage than ordered indexes, so not very common in open-source databases
- Several other kinds of indexes as well:
  - Bitmap indexes – to speed up queries on multiple keys
    - Also less common in open-source databases
  - R-tree indexes – to make spatial queries very fast
    - With ubiquity of geospatial data, quite common these days

RMIT UNIVERSITY

34

34

Index Implementations – B+ Tree

- A *very* widely used ordered index storage format
- Manages a balanced tree structure
  - Every path from root to leaf is the same length
  - Generally remains efficient for selects, even with inserts and deletes occurring
- Can consume significant space, since individual nodes can be up to half empty!
- Index updates for insert and delete can be slow.
  - Tree structure must be updated properly
  - Inserts may require one or more nodes to be split
  - Deletes may require one or more nodes to be merged.
- Performance benefits on queries more than outweigh these costs!
- Queries are straightforward: exact match, partial-match, range, etc

RMIT UNIVERSITY

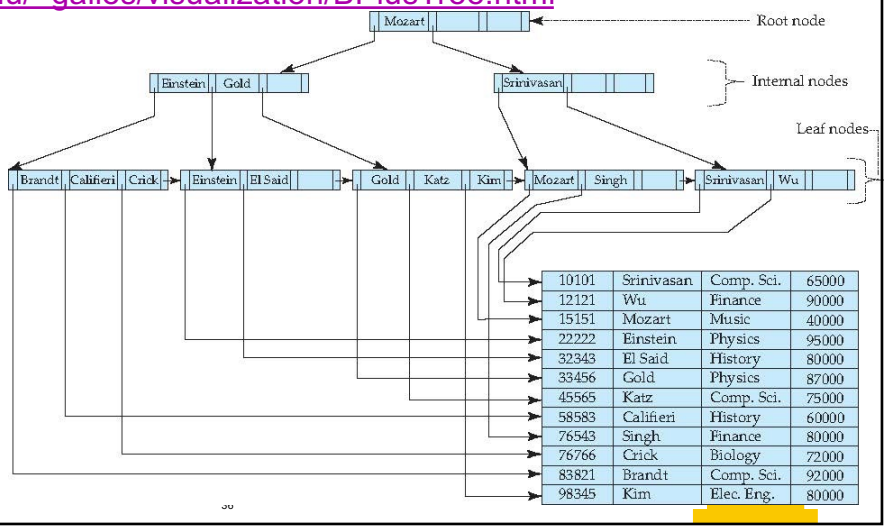
35

35

RMIT Classification: Trusted

Index Implementations – B+ Tree Tree

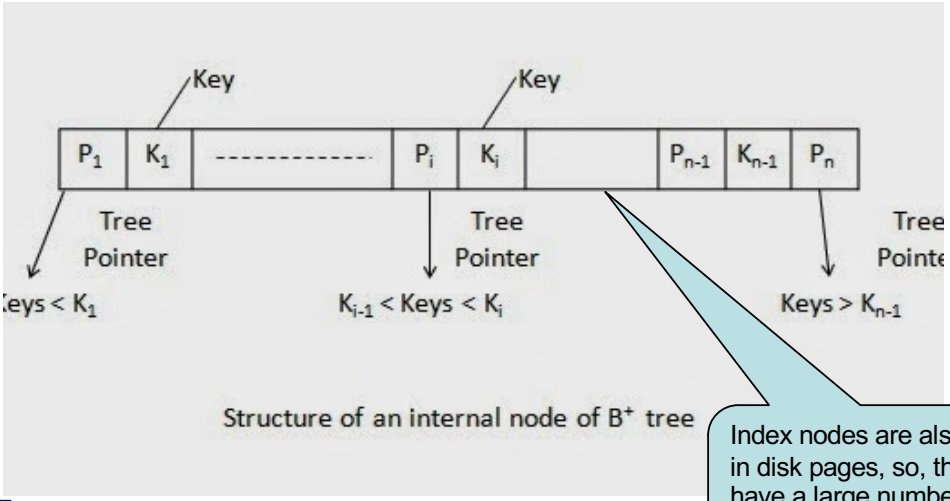
https://www.cs.usfca.edu/~galles/visualization/BPlusTree.html



RMIT UNIVERSITY

36

B+ Trees for Indexing



Index nodes are also stored in disk pages, so, they can have a large number of <key, pointer> pairs.

RMIT UNIVERSITY

37

37

Height of a B+ Tree

- Height = $\text{Log}_f(N)$
  - F – Fanout – how many <key, pointer> pairs in a disk page
  - N – No. of rows in the table
- Not all pages are filled to the maximum – space left in nodes to accommodate future insertions. Normally, on average 2/3 of node utilisation
- Use an online logarithm calculator, such as https://www.calculator.net/log-calculator.html
- E.g. 7 million records
  - key value (integer) = 4 bytes
  - Pointer = 4 bytes
  - Block size = 4096 bytes
  - Load factor = 67% (or 2 third filled on average)
  - Fanout ~ (approx) 343
  - Height = Log 343 (7000000) = 2.7 → rounded up: 3 levels

RMIT UNIVERSITY

38

38

Index Implementations – Hash Index

- A hash index consists of a collection of buckets organized in an array. A hash function maps index keys to corresponding buckets in the hash index.
- Multiple index keys may be mapped to the same hash bucket.
- If two index keys are mapped to the same hash bucket, there is a hash collision. A large number of hash collisions can have a performance impact on read operations.
- The in-memory hash index structure consists of an array of memory pointers.

RMIT UNIVERSITY

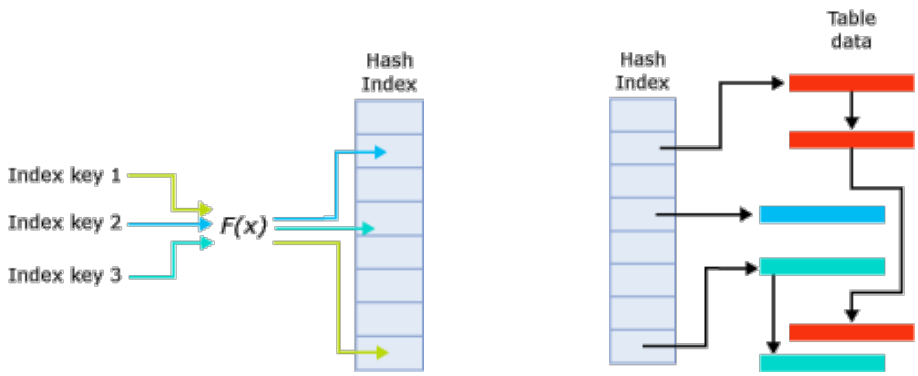
39

39

39

RMIT Classification: Trusted

Index Implementations – Hash Index



RMIT UNIVERSITY

RMIT Classification: Trusted

B+ Tree Index	Hash Index

`CREATE INDEX name ON table(attributes);`	Syntax depend on the DBMS
	`ALTER TABLE <table Name>`
	`ADD INDEX <index name>`
	`HASH (<column name>) WITH (BUCKET_COUNT = <number buckets>);`

Works with all kinds of queries: exact match, partial match, range, etc	Works with exact match only
Generally takes up disk space, and based on the choice of the key, can be large	Small(ish) enough to maintain in-memory, and as a result, very fast
Not sensitive to data spread (even mostly skewed data produce balanced ++ trees.	If (key) data spread is skewed, there can be lot of collisions and as a result, performance can degrade.

41

Verifying Indexing Usage

- Very important to verify that your new index is actually being used!
- Sometimes (cost-based) query optimiser decides against the use of an index! It might have determined a full table scan can be cheaper.
- Consider the following query:

```
SELECT tailnum FROM ontime
   WHERE year >= 2008;
```

- Can you guess why the optimiser made that choice?

RMIT UNIVERSITY

42

42

Indexing – Disadvantages

- Indexes impose an overhead in both space and time
- Speeds up "SELECT"s, but slows down all modifications
- Large keys seriously degrade index performance.
  - Example: B_+ trees
  - Biggest benefit is very large branching factor of each node
  - Large key-values will dramatically reduce the branching factor, deepening the tree and increasing IO costs
- Adding indexes on frequently changing fields
  - Any drawbacks to putting an index on account balances?
  - Account balances change all the time.
  - Will definitely incur a performance penalty on updates

RMIT UNIVERSITY

43

43

Filtered Index – Mitigating Disadvantages

This builds a filtered index on rows related to first 10 months of the ontime data, assuming no new rows are added, this portion of the table is static, hence no additional index management costs.

- Filtered indexing allows us to build an index on a por the table – a portion that is unlikely (or less-likely) change o e and more queries are likely be based on.
- An example is to build an index on older row a transactions table where filtering is done based on transactio ate.

```
CREATE INDEX FilteredOntim ailnumIDX
      ON ontime(tailnum)
      WHERE month <=10;
```

RMIT UNIVERSITY

44

44

Filtered Index – Mitigating Disadvantages

This query does not get any benefit from the index.

- Now try out these two queries

```
SELECT tailnum
    FROM ontime
    WHERE tailnum LIKE 'N305%';
```

```
SELECT tailnum
    FROM ontime
    WHERE month = 1 AND
          tailnum LIKE 'N305%';
```

This query does use index.

RMIT UNIVERSITY

45

45

Next week…

- We will explore how query optimiser does its magic and how can you help with its role.

RMIT UNIVERSITY

46

46

Menti-time!

Menti.com

Code:

RMIT UNIVERSITY

47

Questions?

RMIT UNIVERSITY

48